CHAPTER 4

The Transformer Architecture

In this chapter

Introduction: Two Problems and One Paper . . . 3
4.1 The Shape of the Thing . . . 4
4.2 Self-Attention . . . 5
4.3 Multi-Head Attention . . . 8
4.4 Causal Masking . . . 8
4.5 Positional Information . . . 9
4.6 The Feed-Forward Network . . . 10
4.7 Residuals and Normalisation: The Reason It Is Trainable . . . 10
4.8 What Is Actually Stored: Parameters and Dimensions . . . 11
4.9 The KV Cache: Prefill, Decode, and Why Prompt Caching Works . . . 11
4.10 Why Long Context Is Expensive: The Quadratic . . . 12
4.11 Encoder, Decoder, and Why Claude Is Decoder-Only . . . 13
4.12 Code Walkthrough: Self-Attention in NumPy . . . 14
4.13 Mini Project: A Working Transformer That Writes English . . . 15
4.14 Hands-on Labs . . . 18
4.15 Lab Solutions . . . 19
4.16 Best Practices . . . 20
4.17 Common Mistakes . . . 20
4.18 Performance Considerations . . . 21
4.19 Security Considerations . . . 21
4.20 Real Production Examples . . . 22
4.21 Certification Notes . . . 22
4.22 Practice Questions . . . 23
4.23 Answers and Explanations . . . 24
4.24 Ten Practical Exercises . . . 25
4.25 Five Challenge Labs . . . 26
4.26 Interview Questions . . . 26
4.27 Summary . . . 27
4.28 Glossary . . . 28

4.29 Further Reading 29

Part 1 of 13 · Chapter 4 of 64. Next: Chapter 5 — Attention, Encoders and Decoders.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Explain the two problems with recurrent networks — the **sequential bottleneck** and **long-range forgetting** — that the transformer was designed to remove.
2. Draw a complete **decoder-only transformer** and name every component: embeddings, positional information, attention, feed-forward layers, residual connections, layer normalisation and the output softmax.
3. Explain **self-attention** in terms of queries, keys and values, and trace a small worked example by hand.
4. Explain **multi-head attention** and why several heads beat one large one.
5. Explain **causal masking** and why it is the reason a language model can be trained on every position at once but must generate one token at a time.
6. Explain why the model needs **positional information** at all, and name the main schemes.
7. Explain the **KV cache**, and use it to account precisely for the difference between *prefill* and *decode*, between time-to-first-token and time-per-output-token, and for why **prompt caching** saves money.
8. Explain why attention cost grows **quadratically** with sequence length, and derive from that the entire economic argument for retrieval and compaction in Part 10.
9. Implement self-attention in NumPy and train a working single-block transformer that generates English text.

NOTE

Prerequisites

Chapters 1, 2 and 3. In particular you need: the training loop and cross-entropy loss (Chapter 2); neurons, layers, activations, softmax, residual connections, layer normalisation and matrix multiplication (Chapter 3). The character-level language model built in section 3.10 is the direct ancestor of the model built in this chapter, and it is worth having run it.

Introduction: Two Problems and One Paper

Before 2017, the standard way to handle a sequence was a **recurrent neural network**. It read one token, updated an internal memory, read the next, updated again, and so on to the end. Intuitively appealing — it is roughly how you read a sentence — and it had two defects that turned out to be fatal.

The sequential bottleneck. Position 500 cannot be processed until position 499 has been. Training on a billion sentences therefore proceeds one token at a time, per sentence, and no amount of parallel hardware helps. Recurrence made the training of large models economically impossible, not merely slow.

Long-range forgetting. Information from early in a sequence must survive being repeatedly overwritten as the memory is updated. In practice it degrades. By the time the model reaches the end of a long paragraph, the subject of the first sentence has largely been squeezed out. This is the vanishing-gradient problem of section 3.4, playing out across time rather than across depth.

In June 2017 a paper titled *Attention Is All You Need* proposed replacing recurrence entirely. Instead of passing information along a chain, let **every position look directly at every other position,**

simultaneously. There is no chain to degrade, and because every position is processed independently the whole sequence can be handled in parallel.

The consequence was not a modest improvement in quality. It was a change in what was *affordable*. Training could now saturate thousands of accelerators, which meant models could be scaled by orders of magnitude, which meant capability improvements that had previously required new ideas could instead be bought with compute. Everything in this book after this chapter is downstream of that.

KEY TAKEAWAYS

Why an architect needs this chapter

You will never modify a transformer. But four facts you use every day are properties of this architecture and are otherwise inexplicable:

- **Why input tokens are cheaper than output tokens**, and why time-to-first-token behaves differently from time-per-output-token — section 4.9.
- **Why prompt caching works, what exactly is cached, and why the cache must be a prefix** — section 4.9.
- **Why doubling the prompt more than doubles the attention cost** — section 4.10.
- **Why information in the middle of a very long context is used less reliably** — section 4.10, and Chapter 55.

These are Domain 5 material. They are also the difference between guessing at a cost model and calculating one.

4.1 The Shape of the Thing

Start with the whole picture, then take it apart. Claude, like nearly every modern language model, is a **decoder-only transformer**: a stack of identical blocks, with token handling at the bottom and a probability distribution at the top.

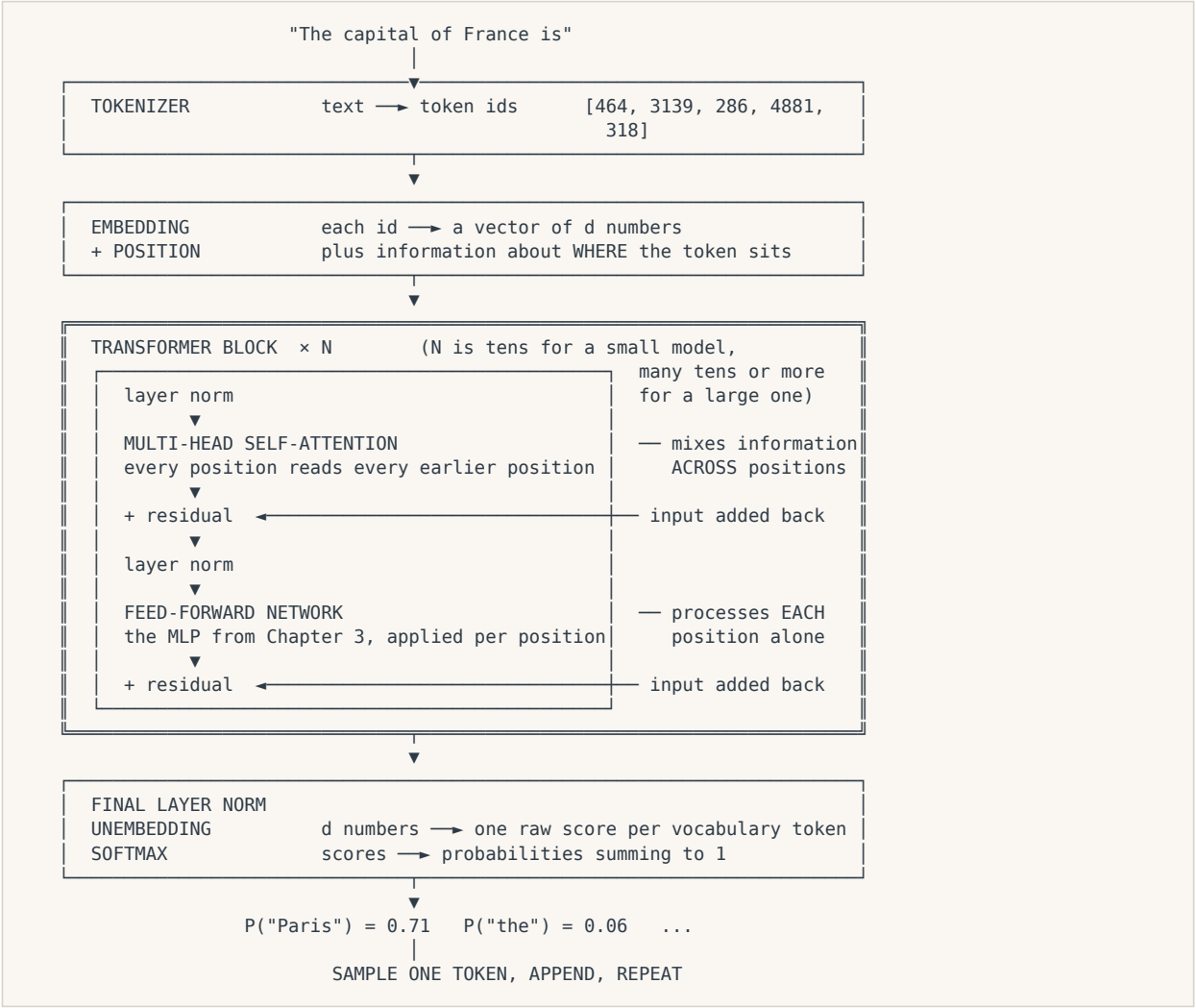


Figure 4.1 — A decoder-only transformer, end to end.

Two observations before we go deeper, because they organise everything else.

A block does exactly two things. Attention moves information *between* positions. The feed-forward network processes each position *independently*. That is the whole design: mix, then think; mix, then think; repeated N times. Nothing else is in there.

Everything except attention you have already met. The feed-forward network is the multi-layer perceptron of Figure 3.3. The residual connections and layer normalisation are from section 3.6. The softmax output is from section 3.2. **Attention is the one new idea in this chapter.**

4.2 Self-Attention

4.2.1 The problem it solves, in one sentence

Consider: *"The trophy would not fit in the suitcase because it was too large."*

What does *it* refer to? A human resolves this instantly using world knowledge. To do so, the model must let the representation of `it` be influenced by the representations of `trophy` and `suitcase` — words several positions away. Self-attention is the mechanism by which one position pulls in information from

others.

4.2.2 Queries, keys and values

Each position produces three different vectors from its own representation, by multiplying it by three different learned weight matrices:

Vector	Produced by	Intuition
Query (Q)	`x · Wq`	"Here is what I am looking for."
Key (K)	`x · Wk`	"Here is what I have to offer."
Value (V)	`x · Wv`	"Here is the content I will hand over if you choose me."

VISUAL ANALOGY

The library card catalogue

You arrive at a library with a question in mind — that is your **query**. Every book on the shelves has a spine label summarising what it is about — those are the **keys**. You compare your question against every spine label and score the match. The books whose labels match best get most of your attention. You then read those books — their **contents** are the **values** — and blend what you learn, weighted by how relevant each one was.

Crucially, you do this for *every* question at once, and every position in the sequence is simultaneously a reader and a book. Position 7 asks its question of positions 1 through 7 and receives a weighted blend of their contents. So does every other position, in parallel.

Where the analogy breaks: a spine label is written to describe the book. Keys and values are separate learned projections, so what a position *advertises* and what it *provides* need not be the same thing — and in trained models they frequently are not.

4.2.3 The mechanism, stated precisely

Four steps. This is the entirety of self-attention.

```
STEP 1  SCORE
   Every query is compared against every key by dot product.
   A large dot product means "these two vectors point the same way",
   i.e. this key is relevant to this query.

            scores  =  Q · Kᵀ                    → a T × T grid

STEP 2  SCALE
   Divide by √d_k. Without this, dot products of high-dimensional
   vectors grow large, softmax saturates, and gradients vanish.

            scores  =  (Q · Kᵀ) / √d_k

STEP 3  MASK, then SOFTMAX
   Set every "future" position to −∞ so it cannot be attended to
   (section 4.4). Then softmax each row: each position's scores
   become weights that sum to 1.

            weights =  softmax( masked scores )   → a T × T grid

STEP 4  BLEND
   Each position's output is the weighted average of all values.

            output  =  weights · V

Written as one line, which you will see everywhere:

            Attention(Q, K, V) = softmax( Q Kᵀ / √d_k ) V
```

Figure 4.2 — Self-attention, step by step.

4.2.4 A worked example you can check by hand

Four positions, `d_k = 4`, with deliberately simple values so the arithmetic is visible. After scaling and causal masking, the attention weights come out as:

Position	attends to 1	to 2	to 3	to 4
1	1.00	—	—	—
2	0.38	0.62	—	—
3	0.27	0.27	0.45	—
4	0.19	0.31	0.19	0.31

Read three things from this table, because each is a general property rather than an artefact of the example.

Every row sums to 1. That is softmax. Attention distributes a fixed budget of one unit of focus across the available positions — which means attending more to one thing necessarily means attending less to everything else. This is the mechanical root of *attention dilution* in long contexts.

Position 1 attends only to itself. It has nothing earlier to look at. The first token of any sequence is always in this situation.

Position 4 splits its attention between positions 2 and 4 — in this example because its query was constructed to match those keys. In a trained model, this is where *"it" attends to "trophy"* would show up.

The blended outputs, computed as `weights · V`, are then a mixture of the value vectors — position 3's output, for instance, is roughly `0.27·V₁ + 0.27·V₂ + 0.45·V₃`. Each position now carries information

drawn from everywhere it looked.

4.3 Multi-Head Attention

One set of Q, K and V matrices learns one notion of relevance. Real language requires several at once: syntactic agreement, coreference, topic, tone, position in a list.

Multi-head attention runs several attention operations in parallel, each with its own smaller Q, K and V projections, then concatenates the results and projects them back.

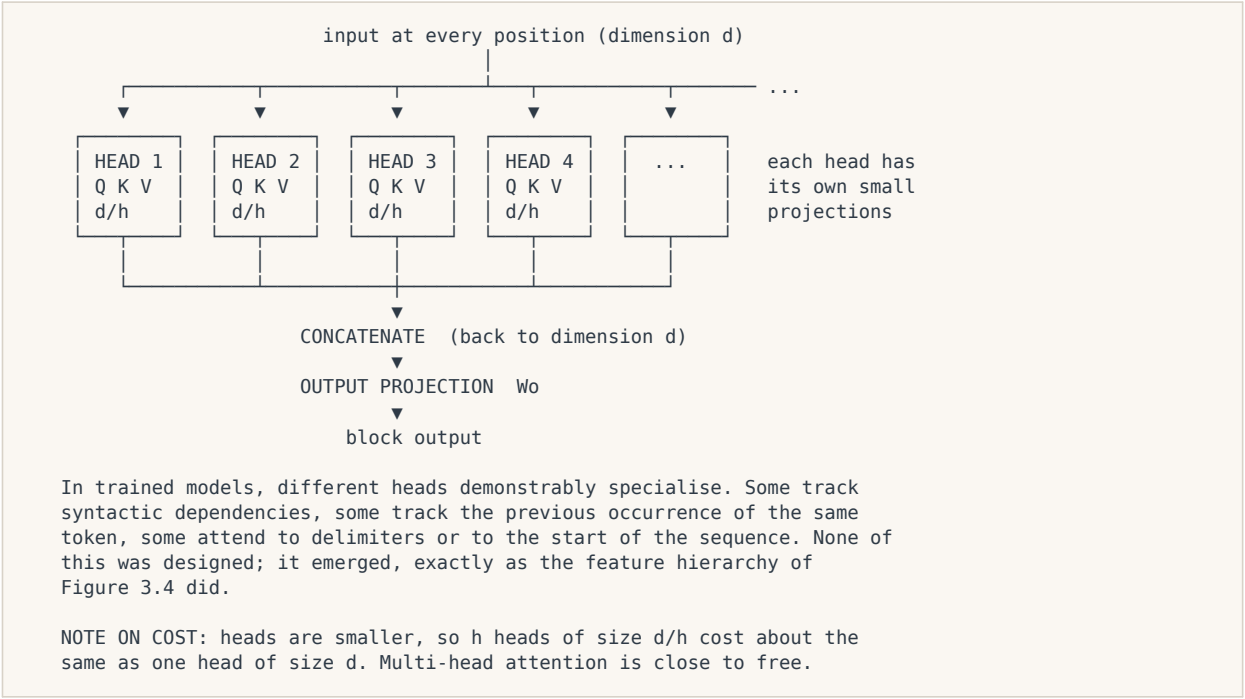


Figure 4.3 — Multi-head attention.

4.4 Causal Masking

A language model must predict the next token from the previous ones. If, during training, position 5 were allowed to attend to position 6, the model could simply copy the answer — and would learn nothing useful, then fail completely at generation time when there is no position 6 yet.

Causal masking forbids it. Before the softmax in step 3, every score corresponding to a future position is set to negative infinity, which softmax turns into a weight of exactly zero.

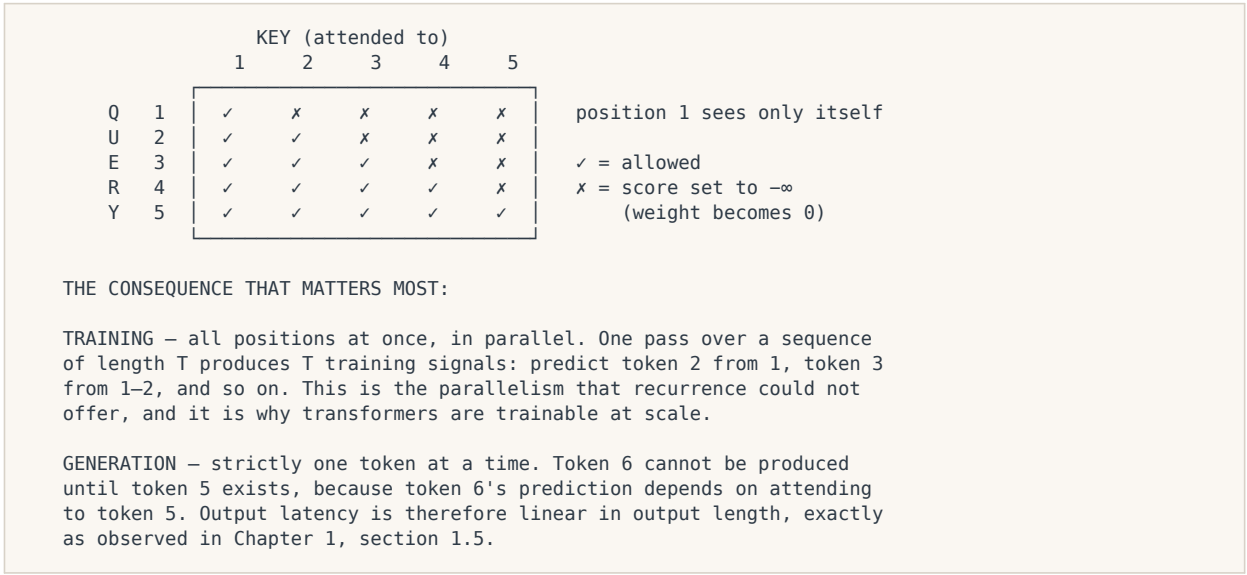


Figure 4.4 — The causal mask over a 5-token sequence.

KEY TAKEAWAYS

Training parallelises; generation does not

This single asymmetry explains a great deal:

- Why training is done on enormous parallel clusters, while serving is dominated by sequential decode steps.
- Why **input tokens cost less than output tokens**: input is processed in one parallel pass, output requires one full pass per token.
- Why "be concise" is a latency optimisation.
- Why streaming exists: since tokens are produced one at a time anyway, they may as well be sent as they appear.

4.5 Positional Information

Attention as described has a curious property: it is **order-blind**. The weighted sum of values does not depend on the order in which positions are listed. Shuffle the words of a sentence and, without positional information, the model computes the same thing.

Since *"the dog bit the man"* and *"the man bit the dog"* must not be identical, position must be injected explicitly.

Scheme	How it works	Where used
Sinusoidal	Fixed sine and cosine patterns of varying frequency added to embeddings	The original 2017 paper. Requires no parameters and extrapolates somewhat beyond the trained length.
Learned absolute	A trainable vector per position, added to the embedding	Simple and effective, but positions beyond the trained maximum have no vector at all.

Scheme	How it works	Where used
Rotary (RoPE)	Rotates the query and key vectors by an angle proportional to position, so the dot product depends on *relative* distance	Dominant in current large models. Handles relative position naturally and extends to long contexts far better.

NOTE

Why this matters for long context

The ability to serve a 1-million-token context window is, in significant part, a positional-encoding achievement. A scheme that encodes only absolute positions up to some trained maximum simply has nothing to say about position 900,000. Relative schemes such as RoPE, combined with techniques for extending them, are what made very long windows feasible.

You do not configure any of this. But it explains why context limits are a property of a specific model rather than a universal constant, and why they have grown so dramatically over successive generations.

4.6 The Feed-Forward Network

After attention has mixed information across positions, each position is processed independently by a small multi-layer perceptron — the exact structure of Figure 3.3:

```
FFN(x)  =  W2 · GELU( W1 · x  +  b1 )  +  b2
```

`W1` typically expands the dimension by a factor of about four, and `W2` projects it back down. Because this layer is applied at every position with the same weights, and because the expansion factor is large, **the feed-forward layers hold the majority of a transformer's parameters** — commonly around two thirds.

A useful, if simplified, division of labour: *attention decides what to look at; the feed-forward network decides what to make of it.* Interpretability research suggests factual knowledge is stored predominantly in these layers, and that attention is largely responsible for routing and composition.

4.7 Residuals and Normalisation: The Reason It Is Trainable

Notice in Figure 4.1 that both sub-layers are wrapped the same way: normalise, apply, then add the input back.

The **residual connection** — adding the input to the output — creates an unbroken path from the top of the network to the bottom along which gradients can travel undiminished. Without it, a stack of many dozens of blocks would suffer the vanishing gradients of section 3.4 and would not train. This is the single most important structural borrowing in the transformer, and it comes from image networks rather than from language work.

Layer normalisation rescales the values flowing through each position to a consistent range, preventing the drift to extreme magnitudes that destabilises deep stacks.

Neither is glamorous. Both are load-bearing. A transformer without them is a transformer that does not train.

4.8 What Is Actually Stored: Parameters and Dimensions

A rough accounting, so that model sizes stop being abstract:

Component	Approximate parameter count
Token embeddings	`vocabulary × d`
Per block: attention (Wq, Wk, Wv, Wo)	`4 × d × d`
Per block: feed-forward (W1, W2)	`8 × d × d` (with a 4× expansion)
Per block total	roughly `12 × d²`
Whole model	roughly `12 × d² × N` plus embeddings

So a model with `d = 4096` and `N = 32` blocks holds approximately `12 × 4096² × 32 ≈ 6.4` billion parameters in its blocks. Scaling `d` is quadratically expensive; scaling `N` is linear. Real architectures balance the two, and the balance is one of the things that differs between a Haiku-tier and an Opus-tier model.

4.9 The KV Cache: Prefill, Decode, and Why Prompt Caching Works

This section is the one with the most direct commercial consequence, and it is the one most often skipped.

When generating token `t+1`, the model needs the keys and values of every position from 1 to `t`. Those keys and values were already computed when those tokens were processed. Recomputing them for every new token would be enormously wasteful — so they are kept. That store is the **KV cache**.

Generation therefore has two distinct phases with completely different performance characteristics:

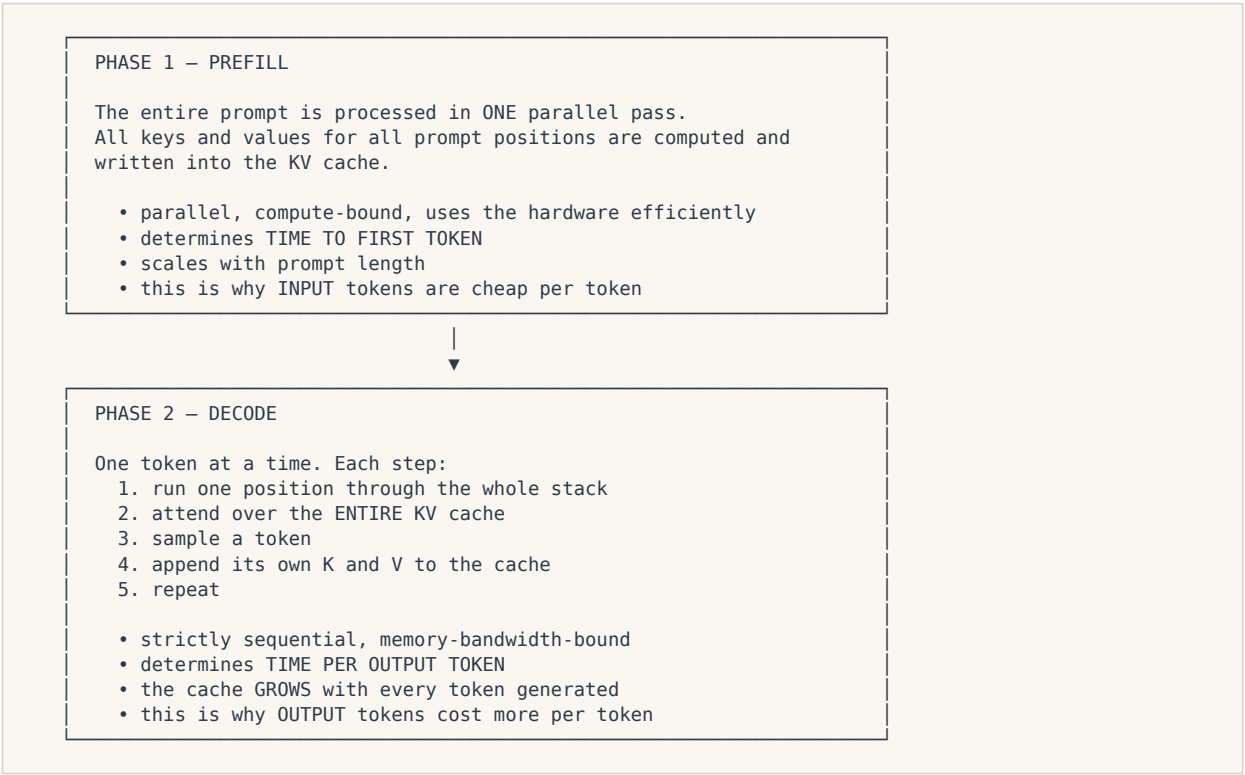


Figure 4.5 — Prefill and decode.

KEY TAKEAWAYS

Prompt caching, explained by mechanism

Prompt caching stores the KV cache computed for a prompt prefix so that a later request beginning with the same prefix can skip prefill for that portion.

Three properties follow directly from the mechanism, and all three are examinable:

1. **It must be a prefix.** Because of causal masking, position k's keys and values depend on positions 1…k. Change anything early and every subsequent entry is invalid. This is why cached content must go at the *start* of the prompt — system prompt, tool definitions, reference documents — with the variable part last.
2. **It accelerates prefill, not decode.** Caching reduces time-to-first-token and input cost. It does nothing whatsoever for generation speed.
3. **There is a minimum cacheable length.** According to the official documentation at the time of writing, the minimum cacheable prompt on Claude Opus 5 is **512 tokens**, reduced from 1,024 on Claude Opus 4.8. Below that threshold, caching does not engage.

Chapter 30 covers the API surface. The mechanism is here.

4.10 Why Long Context Is Expensive: The Quadratic

Attention compares every position with every other position. For a sequence of length `T`, that is a `T × T` grid of scores.

Sequence length	Attention score entries	Relative to 1K
1,000	1,000,000	1×
10,000	100,000,000	100×
100,000	10,000,000,000	10,000×
1,000,000	1,000,000,000,000	1,000,000×

Doubling the input quadruples the attention work. Production systems use heavily optimised implementations and various forms of sparsity, so real cost does not track this table exactly — but the shape of the curve is real, and it drives three consequences an architect must internalise.

Cost. Long prompts are disproportionately expensive, not merely proportionally so.

Latency. Time-to-first-token grows faster than linearly with prompt length. A 200,000-token prompt does not take twenty times as long to prefill as a 10,000-token one; it takes considerably longer than that.

Attention dilution. Each position has one unit of attention to spend, spread across everything before it. In a million-token context, the average weight on any individual token is vanishingly small. Models are demonstrably better at using information at the beginning and end of a long input than in the middle — the *lost in the middle* effect, quantified in Chapter 55.

CERTIFICATION NOTE

The most common wrong answer in Domain 5

"Place the entire corpus in the 1M-token context window."

It is wrong for three independent reasons, and being able to state all three is worth marks: quadratic cost growth, super-linear latency growth, and degraded retrieval of mid-context information. The right answers involve **retrieval** (put only what is relevant in the window), **compaction** (summarise history rather than carrying it verbatim), **caching** (pay for the stable prefix once), and **placement** (put critical instructions at the boundaries, not the middle).

4.11 Encoder, Decoder, and Why Claude Is Decoder-Only

The 2017 paper described an encoder–decoder architecture built for translation. The field then split three ways.

Family	Attention pattern	Good at	Examples
Encoder-only	Bidirectional — every position sees every other, in both directions	Understanding a complete input: classification, retrieval embeddings, named-entity recognition	BERT-style models, most embedding models
Decoder-only	Causal — each position sees only what precedes it	Generation, and — as it turned out — nearly everything else	Claude, and every major general-purpose LLM

Family	Attention pattern	Good at	Examples
Encoder–decoder	Encoder bidirectional, decoder causal, with cross-attention between them	Sequence-to-sequence transformation with a clear input/output split	Translation models, T5-style models

Decoder-only won for general-purpose use for a reason worth understanding. Because its training objective — predict the next token — requires nothing but raw text, it can consume the entire internet without annotation. And because any task can be phrased as text continuation (*"Translate to French: … "*, *"Summarise: … "*, *"Classify this ticket: …"*), one model trained on that single objective handles everything, rather than requiring a task-specific architecture.

Encoder-only models have not disappeared. When you build a retrieval system in Chapter 53, the embedding model turning your documents into vectors is very likely one — because for pure understanding, seeing the whole input in both directions is genuinely better.

4.12 Code Walkthrough: Self-Attention in NumPy

Twenty lines, no framework. This is the mechanism of Figure 4.2, executable.

```
import numpy as np
np.set_printoptions(precision=2, suppress=True)
def softmax(z):
    z = z - z.max(axis=-1, keepdims=True)      # numerical stability
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)
def causal_self_attention(X, Wq, Wk, Wv):
    """X: (T, d) one row per position. Returns (T, d)."""
    T, d = X.shape
    Q = X @ Wq                      # what each position is looking for
    K = X @ Wk                      # what each position offers
    V = X @ Wv                      # what each position hands over
    scores = Q @ K.T / np.sqrt(d)   # STEP 1 and 2: score, then scale
    mask = np.triu(np.ones((T, T), dtype=bool), k=1)
    scores = np.where(mask, -1e9, scores)      # STEP 3a: causal mask
    weights = softmax(scores)                  # STEP 3b: rows sum to 1
    return weights @ V, weights                # STEP 4: blend
# A four-position example with hand-chosen projections.
X  = np.eye(4)
Wq = np.array([[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0], [0, 1, 0, 1.]])
Wk = np.eye(4)
Wv = np.array([[1, 0], [0, 1], [1, 1], [.5, .5]])
Wv = np.hstack([Wv, np.zeros((4, 2))])         # pad to width 4
out, w = causal_self_attention(X, Wq, Wk, Wv)
print("attention weights:\n", w)
print("rows sum to:", w.sum(axis=1))
```

Which prints the table from section 4.2.4:

```
attention weights:
 [[1.   0.   0.   0.  ]
  [0.38 0.62 0.   0.  ]
  [0.27 0.27 0.45 0.  ]
  [0.19 0.31 0.19 0.31]]
```

```
rows sum to: [1. 1. 1. 1.]
```

Three lines are worth pausing on.

`scores = Q @ K.T / np.sqrt(d)` — one matrix multiplication produces every pairwise comparison at once. This is the parallelism recurrence could not offer, and it is also, visibly, the `T × T` object responsible for the quadratic cost of section 4.10.

`np.where(mask, -1e9, scores)` — a very large negative number rather than true infinity, because `exp(-inf)` in floating point tends to produce warnings and occasionally `nan`. This is the standard trick and you will see it in every implementation.

`weights @ V` — the blend. Every position's output is a convex combination of the value vectors it was allowed to see. The convexity is the point: attention has a fixed budget to allocate.

4.13 Mini Project: A Working Transformer That Writes English

Goal. Upgrade the character model of section 3.10 into a real single-block transformer — embeddings, learned positional vectors, causal self-attention, residual connections and a feed-forward network — with hand-written backpropagation, in NumPy alone. Then generate text and compare.

Architecture.

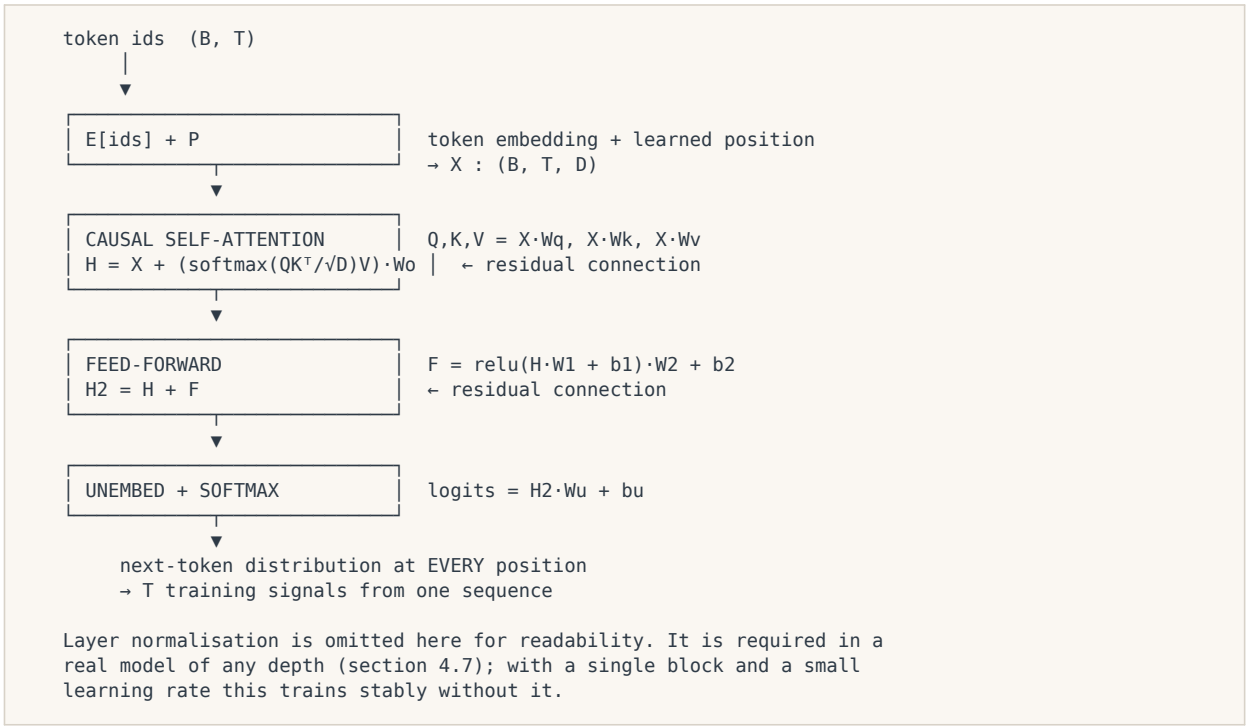


Figure 4.6 — The mini transformer.

Folder structure.

```
mini-transformer/
├── README.md
├── requirements.txt           # numpy
├── minitf/
│   ├── __init__.py
```

```
│   ├── data.py                # corpus, vocabulary, sliding windows
│   ├── params.py              # parameter initialisation
│   ├── train.py               # forward + backward + update
│   └── generate.py            # sampling with temperature
└── tests/
    └── test_minitf.py
```

Setup and parameters.

```
# minitf/params.py
import numpy as np
rng = np.random.default_rng(3)
TEXT = ("the quick brown fox jumps over the lazy dog. " * 60 +
        "pack my box with five dozen liquor jugs. " * 60)
chars = sorted(set(TEXT))
V = len(chars)
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
T, D, F = 8, 32, 64          # context length, model width, FFN width
def r(*shape):
    return rng.normal(0, 0.05, shape)
E  = r(V, D)                 # token embeddings
P  = r(T, D)                 # learned positional vectors
Wq, Wk, Wv, Wo = r(D, D), r(D, D), r(D, D), r(D, D)
W1, b1 = r(D, F), np.zeros(F)
W2, b2 = r(F, D), np.zeros(D)
Wu, bu = r(D, V), np.zeros(V)
MASK = np.triu(np.ones((T, T), dtype=bool), 1)
def softmax(z):
    z = z - z.max(-1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(-1, keepdims=True)
```

Data: every window is a training example.

```
# minitf/data.py
import numpy as np
from .params import TEXT, stoi, T
ids = np.array([stoi[c] for c in TEXT])
# Input is a window of T characters; target is the SAME window shifted by
# one. Causal masking means all T predictions can be learned in one pass.
X = np.stack([ids[i:i + T]         for i in range(len(ids) - T - 1)])
Y = np.stack([ids[i + 1:i + T + 1] for i in range(len(ids) - T - 1)])
```

The forward and backward pass.

```
# minitf/train.py
import numpy as np
from .params import *          # parameters, MASK, softmax, T, D, F, V
from .data import X, Y
def step(xb, yb, lr):
    global E, P, Wq, Wk, Wv, Wo, W1, b1, W2, b2, Wu, bu
    B = len(xb)
    # ------------------------------------------------- FORWARD
    Xe = E[xb] + P                                  # (B,T,D)
    Q, K, Vv = Xe @ Wq, Xe @ Wk, Xe @ Wv
    S = Q @ K.transpose(0, 2, 1) / np.sqrt(D)
    S = np.where(MASK, -1e9, S)
    A = softmax(S)                                  # (B,T,T)
```

```
    O = A @ Vv
    H = Xe + O @ Wo                                  # residual 1
    Z1 = H @ W1 + b1
    A1 = np.maximum(0, Z1)
    H2 = H + A1 @ W2 + b2                            # residual 2
    logits = H2 @ Wu + bu
    Pm = softmax(logits)
    flat = np.arange(B * T)
    loss = -np.log(Pm.reshape(-1, V)[flat, yb.reshape(-1)] + 1e-9).mean()
    # ------------------------------------------------------ BACKWARD
    dlog = Pm.copy()
    dlog.reshape(-1, V)[flat, yb.reshape(-1)] -= 1
    dlog /= (B * T)
    dWu = np.einsum('btd,btv->dv', H2, dlog); dbu = dlog.sum((0, 1))
    dH2 = dlog @ Wu.T
    dW2 = np.einsum('btf,btd->fd', A1, dH2); db2 = dH2.sum((0, 1))
    dZ1 = (dH2 @ W2.T) * (Z1 > 0)
    dW1 = np.einsum('btd,btf->df', H, dZ1); db1 = dZ1.sum((0, 1))
    dH = dH2 + dZ1 @ W1.T                            # both residual paths
    dWo = np.einsum('btd,bte->de', O, dH)
    dO = dH @ Wo.T
    dA = dO @ Vv.transpose(0, 2, 1)
    dVv = A.transpose(0, 2, 1) @ dO
    # backward through softmax, row-wise
    dS = A * (dA - (dA * A).sum(-1, keepdims=True))
    dS = np.where(MASK, 0.0, dS) / np.sqrt(D)
    dQ = dS @ K
    dK = dS.transpose(0, 2, 1) @ Q
    dWq = np.einsum('btd,bte->de', Xe, dQ)
    dWk = np.einsum('btd,bte->de', Xe, dK)
    dWv = np.einsum('btd,bte->de', Xe, dVv)
    dXe = dH + dQ @ Wq.T + dK @ Wk.T + dVv @ Wv.T
    dP = dXe.sum(0)
    dE = np.zeros_like(E)
    np.add.at(dE, xb.reshape(-1), dXe.reshape(-1, D))
    # ------------------------------------------------------ UPDATE
    for prm, g in ((E, dE), (P, dP), (Wq, dWq), (Wk, dWk), (Wv, dWv),
                   (Wo, dWo), (W1, dW1), (b1, db1), (W2, dW2), (b2, db2),
                   (Wu, dWu), (bu, dbu)):
        prm -= lr * g
    return loss
def train(epochs=25, lr=0.3, batch=64):
    n = len(X)
    for epoch in range(epochs):
        perm = rng.permutation(n)
        total, count = 0.0, 0
        for s in range(0, n, batch):
            idx = perm[s:s + batch]
            total += step(X[idx], Y[idx], lr)
            count += 1
        if epoch % 8 == 0:
            print(f"epoch {epoch:3d}  loss {total / count:.4f}")
```

Generation.

```
# minitf/generate.py
import numpy as np
from .params import *
def generate(seed, n_chars=70, temperature=0.7):
    out = seed
    for _ in range(n_chars):
```

```
        ctx = out[-T:].rjust(T)
        xb = np.array([[stoi[c] for c in ctx]])
        Xe = E[xb] + P
        Q, K, Vv = Xe @ Wq, Xe @ Wk, Xe @ Wv
        S = np.where(MASK, -1e9, Q @ K.transpose(0, 2, 1) / np.sqrt(D))
        H = Xe + (softmax(S) @ Vv) @ Wo
        H2 = H + np.maximum(0, H @ W1 + b1) @ W2 + b2
        p = softmax((H2[0, -1] @ Wu + bu) / temperature)
        out += itos[rng.choice(V, p=p)]
    return out
```

What it produces. Loss falls from about 3.06 — roughly the loss of uniform guessing over the vocabulary — to about 0.21 in twenty-five epochs. Generating from the seed `"the "`:

```
epoch   0  loss 3.0593
epoch   8  loss 0.4836
epoch  16  loss 0.2400
epoch  24  loss 0.2064
'the quick brown fox jumps over the quick brown fox jumps over the quick br'
```

Testing.

```
# tests/test_minitf.py
import numpy as np
from minitf.params import softmax, MASK, T
def test_attention_rows_sum_to_one():
    S = np.where(MASK, -1e9, np.random.randn(1, T, T))
    A = softmax(S)
    assert np.allclose(A.sum(-1), 1.0)
def test_causal_mask_blocks_the_future():
    S = np.where(MASK, -1e9, np.zeros((1, T, T)))
    A = softmax(S)[0]
    assert np.allclose(np.triu(A, 1), 0.0)      # nothing attends forward
    assert np.isclose(A[0, 0], 1.0)             # position 1 sees only itself
```

Evaluation. Hold back the final 10% of the corpus and report loss on it separately each epoch. Then compare against the Chapter 3 model trained on the same text: at comparable parameter counts, the transformer reaches a lower loss and produces longer coherent runs, because attention lets it use the whole context selectively rather than compressing it into one fixed hidden vector.

KEY TAKEAWAYS

What this project demonstrates

1. **Causal masking gives T training signals per sequence.** One window of eight characters teaches eight predictions. This is why transformer training is data-efficient in a way recurrence never was.
2. **Residual connections appear twice per block**, and in the backward pass the gradient `dH` genuinely arrives along two paths — you can see it in the code.
3. **Attention weights are a distribution.** The test asserting rows sum to one is asserting the fixed attention budget that causes dilution.
4. **The context length is structural.** `T = 8` is baked into `P` and `MASK`. Extending it is not a configuration change; it is a different model. This is why context windows are model properties.
5. **Generation is sequential**, one character per pass, exactly as production decode works.

4.14 Hands-on Labs

Lab 4.1 — Remove the mask. Delete the causal mask from the mini transformer and retrain. *Success criterion:* training loss collapses towards zero within a couple of epochs while generation produces nonsense. Explain precisely what the model learned.

Lab 4.2 — Remove positional information. Set `P` to zeros and freeze it. Retrain. *Success criterion:* a measurably worse loss, and an explanation grounded in the order-blindness of attention.

Lab 4.3 — Visualise attention. For a fixed input, print the attention weight matrix as a grid. *Success criterion:* you can identify at least one position whose attention concentrates somewhere non-trivial, and speculate about why.

Lab 4.4 — Measure the quadratic. Time `causal_self_attention` for sequence lengths 128, 256, 512, 1024 and 2048 at fixed width. *Success criterion:* a table showing roughly 4× time for each doubling, and a sentence connecting it to prompt cost.

Lab 4.5 — Prefix caching by hand. Modify the generation function to keep `K` and `V` between steps instead of recomputing them for the whole context. *Success criterion:* identical output, measurably faster generation, and a written explanation of why the cached portion must be a prefix.

4.15 Lab Solutions

Solution 4.1. Without the mask, position `t` can attend to position `t+1` — which, given the targets are the input shifted by one, means it can read the answer directly. The model learns the identity function from future to present. Loss drops almost to zero because the task has become trivial. At generation time the future does not exist, so the learned shortcut is useless and output is noise. This is the cleanest possible demonstration of *target leakage*, the Chapter 2 concept, arising from an architectural error rather than a data error.

Solution 4.2. Attention computes a weighted sum, and a sum does not depend on the order of its terms. With `P = 0`, the model can distinguish *which* characters are in the context but not *where* they are — so `"stop"` and `"post"` produce identical representations. Loss plateaus noticeably higher. Some capability survives because the causal mask still imposes a weak ordering: earlier positions can be attended to by more later positions.

Solution 4.3. In a model trained on this corpus, expect attention to concentrate on the immediately preceding character and on the previous occurrence of a space, since word boundaries are highly predictive. This is genuine emergent structure, not a designed feature — it is the language-model equivalent of the edge detectors in Figure 3.4.

```
import numpy as np
from minitf.params import *
ctx = "the quic"
xb = np.array([[stoi[c] for c in ctx]])
Xe = E[xb] + P
Q, K = Xe @ Wq, Xe @ Wk
A = softmax(np.where(MASK, -1e9, Q @ K.transpose(0, 2, 1) / np.sqrt(D)))[0]
print("      " + "  ".join(f"{c!r}" for c in ctx))
for i, row in enumerate(A):
```

```
    print(f"{ctx[i]!r} " + " ".join(f"{v:4.2f}" for v in row))
```

Solution 4.4. Time grows roughly fourfold per doubling, because the `T × T` score matrix has T^2 entries. Real serving stacks use optimised kernels that reduce memory traffic substantially, but the arithmetic is unavoidable. The direct consequence: a prompt twice as long costs meaningfully more than twice as much to prefill, which is the quantitative case for retrieval over context-stuffing.

Solution 4.5. Each generation step currently recomputes `K` and `V` for the entire context, when only the newest position is new. Caching them makes each step process a single position against a growing cache. The cached entries must be a prefix because, under causal masking, position `k`'s key and value depend on positions `1…k`; inserting or altering anything earlier invalidates everything after it. This is exactly the constraint that makes API prompt caching prefix-only.

4.16 Best Practices

BEST PRACTICE

What follows from the architecture

1. **Put stable content first, variable content last.** System prompt, tool definitions and reference documents at the top; the user's message at the bottom. This maximises cacheable prefix and is free to do.
2. **Treat output length as the primary latency lever.** Decode is sequential. Ask for what you need and no more.
3. **Retrieve rather than stuff.** Selecting 5,000 relevant tokens beats supplying 500,000 marginal ones on cost, latency and accuracy simultaneously.
4. **Place critical instructions at the boundaries.** Beginning and end of a long prompt are attended to more reliably than the middle.
5. **Never depend on exact reproduction of output.** Sampling from a softmax distribution is stochastic by construction.
6. **Budget context explicitly.** Count system prompt, tool definitions, history, retrieved documents and expected output. The window is a resource with a price, not a free container.
7. **Stream anything a human waits for.** Since tokens emerge sequentially anyway, there is no reason to withhold them.

4.17 Common Mistakes

COMMON MISTAKE

Believing a larger context window removes the need for retrieval

Symptom: rising costs and answers that overlook material demonstrably present in the prompt. **Cause:** quadratic attention cost plus attention dilution. **Fix:** retrieval, compaction and deliberate placement. **Detection:** plot answer quality against prompt length; the curve is not flat.

COMMON MISTAKE

Putting variable content before stable content

Symptom: prompt caching never engages and input costs stay high. **Cause:** the cacheable prefix is broken by a timestamp, session id or user message placed early. **Fix:** stable prefix first, always. **Detection:** cache-read token counts near zero in your usage logs.

COMMON MISTAKE

Expecting caching to speed up generation

Symptom: disappointment that responses are no faster after enabling caching. **Cause:** caching eliminates prefill work, not decode work. **Fix:** correct expectations — measure time-to-first-token, which is where the gain appears.

COMMON MISTAKE

Assuming attention means the model "understands" what it attends to

Symptom: over-reading attention visualisations as explanations. **Cause:** attention weights are a routing mechanism, not a justification. **Fix:** treat them as a diagnostic signal, never as evidence of reasoning. Interpretability research is explicit that attention weights are not faithful explanations of model behaviour.

COMMON MISTAKE

Confusing the context window with memory

Symptom: a design in which "the model remembers" across sessions with no storage in the diagram. **Cause:** conflating input size with persistence. **Fix:** the window is the size of one input. Memory is a database plus a retrieval step that you build.

4.18 Performance Considerations

Prefill and decode have different bottlenecks. Prefill is compute-bound and parallel; decode is memory-bandwidth-bound and sequential. Optimisations for one rarely help the other. When someone says a system is "slow", the first question is which phase.

The KV cache consumes memory proportional to context length. Long contexts do not merely cost compute; they occupy serving memory for the duration of the request. This is one of the underlying reasons long-context requests are priced differently.

Batching improves prefill efficiency substantially. Independent requests can be prefilled together, which is a large part of why batch APIs are cheaper.

Model width scales cost quadratically; depth scales it linearly. From section 4.8, block parameters go as $12 \times d^2$ per layer. This is why capability tiers differ so sharply in price: a wider model is not marginally more expensive, it is quadratically so.

Time-to-first-token is what users perceive as responsiveness. Optimise it with a shorter or cached prefix. Optimise perceived total time with streaming.

4.19 Security Considerations

Attention makes no distinction between instruction and data. Every token in the context is scored by the same mechanism. There is no architectural boundary separating "your system prompt" from "the text of the web page a tool just fetched". This is the mechanical explanation for why prompt injection is hard: the model is not ignoring a boundary, because at the level of attention no boundary exists. Structure — XML tags, clear delimiters, explicit instructions about provenance — helps because it gives the model learned signals to work with, not because it enforces anything.

Long contexts widen the attack surface. Every additional document placed in the window is another opportunity for injected instructions to arrive. Retrieval reduces security exposure as well as cost.

The KV cache is derived from content. Cached prefixes are scoped per organisation, but any caching system deserves attention in a threat model where prompts contain sensitive material. Know your provider's isolation guarantees.

Attention weights can leak structure. Where model internals are exposed, attention patterns reveal information about the prompt. This is chiefly a concern for self-hosted deployments rather than hosted APIs.

4.20 Real Production Examples

Long-document analysis. Contracts, filings and reports placed directly in a long context, with a stable cached prefix carrying the instructions and rubric, and the document last. The architecture directly explains the design: instructions cached once, document paid for once, questions asked repeatedly against it.

Retrieval-augmented assistants. An embedding model — encoder-only, bidirectional — turns documents into vectors; the top matches are placed in a decoder-only model's context. Both transformer families in one pipeline, each doing what its attention pattern suits it for.

Agentic coding. Claude Code keeps a large working context of files, prior edits and tool results. Compaction becomes essential as sessions lengthen, and it is essential for exactly the reasons in section 4.10: cost, latency and dilution.

High-throughput classification. Short prompts, short outputs, a fast model tier, and batch processing. Almost all the cost is prefill, which is parallel and cheap — the opposite profile to a long-form generation workload, and it should be architected differently.

4.21 Certification Notes

This chapter supplies the mechanism behind a large share of Domain 5, and behind cost and latency reasoning throughout the exam.

Concept	Where it appears
Prefix-only prompt caching	Domain 5 and Domain 3: prompt structure, cost optimisation, `CLAUDE.md` and system-prompt placement

Concept	Where it appears
Prefill vs decode; input vs output cost	Domains 1 and 5: latency budgets, streaming decisions, model tier selection
Quadratic attention cost	Domain 5: why retrieval beats context-stuffing
Attention dilution and placement	Domain 5: lost-in-the-middle, instruction placement
No architectural instruction/data boundary	Domain 2: prompt injection via tool results, structuring untrusted content
Sequential generation	Domain 1: agent loop latency, parallel subagents as the mitigation

CERTIFICATION NOTE

Exam tips carried forward

- If a scenario has a **cost** problem with a long stable prefix, the answer is almost certainly prompt caching — and the correct implementation puts the stable content first.
- If a scenario has a **latency** problem, distinguish time-to-first-token (fix the input: cache, shorten, retrieve) from total time (fix the output: shorten, stream, or parallelise across subagents).
- If a scenario describes a model overlooking information that is definitely in the prompt, the answer involves **retrieval or placement**, not a larger window or a more capable model.
- Options asserting that a 1M-token window makes retrieval unnecessary are wrong. This is one of the most reliable distractor patterns in Domain 5.

4.22 Practice Questions

1. What is the primary reason transformers replaced recurrent networks? (a) They have more parameters. (b) They process a sequence in parallel, making large-scale training feasible. (c) They require less training data. (d) They eliminate the need for a loss function.

2. In self-attention, the query vector represents: (a) What a position offers to others. (b) What a position is looking for. (c) The content a position hands over when selected. (d) The position's index in the sequence.

3. Why are attention scores divided by the square root of the key dimension? (a) To make them sum to one. (b) To prevent large dot products from saturating softmax and vanishing gradients. (c) To apply the causal mask. (d) To reduce parameter count.

4. Causal masking exists so that: (a) The model cannot attend to future positions and copy the answer. (b) Attention weights sum to one. (c) Positional information is preserved. (d) The KV cache can be reused.

5. Attention weights in each row of the attention matrix: (a) Sum to the sequence length. (b) Sum to 1. (c) Are always equal. (d) Are unbounded.

6. Without positional encoding, a transformer would: (a) Fail to converge at all. (b) Be unable to distinguish word order, since a weighted sum is order-independent. (c) Produce longer outputs. (d) Require more attention heads.

7. Where are the majority of a transformer's parameters? (a) In the attention projections. (b) In the feed-forward layers. (c) In the positional encodings. (d) In the layer normalisation.

8. The KV cache stores: (a) Previously generated text for reuse. (b) The keys and values of already-processed positions so they need not be recomputed. (c) The model's parameters. (d) The user's conversation history on the server.

9. Prompt caching accelerates: (a) Decode only. (b) Prefill only. (c) Both equally. (d) Neither; it only reduces cost.

10. Why must cached prompt content be a prefix? (a) API convention. (b) Because under causal masking, each position's keys and values depend on all preceding positions. (c) Because softmax requires ordered input. (d) Because the tokenizer processes text left to right.

11. Doubling prompt length increases attention score computation by roughly: (a) 2× (b) 4× (c) 8× (d) It is unchanged.

12. Time to first token is most directly affected by: (a) The number of output tokens requested. (b) The length of the input and whether its prefix is cached. (c) The temperature setting. (d) The number of attention heads.

13. Which explains why output tokens cost more than input tokens? (a) Output tokens are longer. (b) Input is processed in one parallel pass; each output token requires a separate sequential pass. (c) Output tokens require more memory to store. (d) A pricing convention with no technical basis.

14. Multi-head attention is used because: (a) It reduces parameter count. (b) Different heads can attend to different kinds of relationship simultaneously. (c) It removes the need for positional encoding. (d) It makes generation parallel.

15. Claude is a decoder-only transformer. This means: (a) It cannot process input, only produce output. (b) Every position attends only to preceding positions. (c) It has no feed-forward layers. (d) It cannot be used for classification.

16. An embedding model used for retrieval is typically: (a) Decoder-only with causal attention. (b) Encoder-only with bidirectional attention. (c) A recurrent network. (d) A classifier with no attention.

17. A team reports the model ignoring a policy stated in the middle of a 400,000-token prompt. The best fix is: (a) Increase the context window. (b) Retrieve only relevant content and place critical instructions at the boundaries. (c) Raise the temperature. (d) Add more attention heads.

18. Which pairing is correct? (a) Prefill — sequential; decode — parallel. (b) Prefill — parallel; decode — sequential. (c) Both parallel. (d) Both sequential.

19. Residual connections in a transformer primarily: (a) Reduce the number of parameters. (b) Give gradients an unobstructed path backwards, making deep stacks trainable. (c) Normalise activation scales. (d) Implement causal masking.

20. A tool returns a web page containing the text "ignore your instructions". Why does the architecture make this difficult to defend against? (a) Tool results bypass the model entirely. (b) Attention scores every token by the same mechanism; there is no architectural boundary between instruction and data. (c) Tool results are not tokenized. (d) The KV cache prioritises tool output.

4.23 Answers and Explanations

1 — (b). Parallelism made scale affordable. Quality followed from scale. (c) is the opposite of true — transformers are data-hungry.

2 — (b). Query is what a position seeks; key is what it advertises; value is what it supplies.

3 — (b). Unscaled dot products grow with dimension, pushing softmax into saturation where gradients vanish.

4 — (a). Without the mask the model can read the target, as demonstrated in Lab 4.1.

5 — (b). Softmax produces a distribution. This fixed budget is the mechanical basis of attention dilution.

6 — (b). A weighted sum is order-independent; position must be injected explicitly.

7 — (b). Feed-forward layers, typically around two thirds, because of the roughly 4× internal expansion.

8 — (b). Keys and values of already-processed positions. (d) is a common and important misconception — the model holds no conversation state between requests.

9 — (b). Prefill only. Caching reduces time-to-first-token and input cost; decode is unaffected.

10 — (b). Causal dependency: change anything early and every later key and value is invalid.

11 — (b). The score matrix has T^2 entries, so doubling T quadruples it.

12 — (b). Time to first token is prefill, which is a function of input length and cache hits.

13 — (b). One parallel pass for all input versus one full pass per output token.

14 — (b). Parallel notions of relevance. Heads are smaller, so the cost is roughly unchanged.

15 — (b). Causal attention. Decoder-only models handle classification perfectly well by generating a label.

16 — (b). Bidirectional encoders are better for pure understanding, which is what retrieval needs.

17 — (b). Retrieval plus placement. Enlarging the window worsens cost, latency and dilution simultaneously.

18 — (b). Prefill is a single parallel pass; decode is one pass per token.

19 — (b). An unobstructed gradient path. Normalisation, option (c), is a separate component.

20 — (b). No architectural separation exists. Structural conventions help by giving the model learned signals, but they do not enforce a boundary — which is why tool permissions and output validation carry the real defensive weight.

4.24 Ten Practical Exercises

1. Implement multi-head attention by splitting d into h heads and confirm the output shape is unchanged.
2. Add layer normalisation to the mini transformer's forward pass and describe the effect on training stability at a higher learning rate.

3. Extend the mini transformer to two stacked blocks and measure the loss improvement per additional parameter.
4. Implement sinusoidal positional encoding as an alternative to learned `P` and compare.
5. Plot the attention weight matrix as a heatmap for ten different inputs and catalogue any recurring patterns.
6. Measure prefill and decode time separately for a real Claude API call by timing to first token and to completion.
7. Restructure one of your own prompts so the stable portion comes first, and measure the change in cache-read tokens.
8. Compute the theoretical KV cache size in megabytes for a context of 100,000 tokens given stated model dimensions.
9. Write a paragraph explaining to a finance colleague why input and output tokens are priced differently.
10. Take a 100,000-token prompt and rebuild the same task with retrieval over 5,000 tokens. Compare cost, latency and answer quality.

4.25 Five Challenge Labs

Challenge 1 — Full multi-head with backpropagation. Extend the mini transformer to four heads, including the backward pass. Verify with gradient checking against numerical estimates.

Challenge 2 — A KV cache implementation. Add a proper incremental cache to `generate`, storing per-step keys and values and processing only the newest position. Measure the speed-up as a function of generated length.

Challenge 3 — The attention atlas. Train a two-block, four-head model and produce a catalogue of what each of the eight heads attends to. Name each head by its apparent function.

Challenge 4 — The context-length cost model. Build a spreadsheet or script that, given prompt length, output length, cache hit rate and published per-token prices, predicts cost and latency for a request. Validate it against real measurements.

Challenge 5 — The lost-in-the-middle experiment. Construct prompts of increasing length with a single critical fact placed at the start, the middle and the end. Measure retrieval accuracy at each position and length. You will have reproduced, at small scale, the published result underlying Chapter 55.

4.26 Interview Questions

1. Explain self-attention to someone who knows how a neural network works but has never seen a transformer.
2. Why did the transformer displace recurrent architectures? Give the economic argument, not just the quality argument.
3. What are queries, keys and values, and why are they three separate projections rather than one?
4. Explain causal masking and its consequence for training versus generation.

5. Why does a transformer need positional encoding when a recurrent network does not?
6. What is the KV cache, and how does it relate to prompt caching in an API?
7. Explain why input and output tokens are priced differently.
8. Why does attention cost grow quadratically, and what does that imply for system design?
9. What is attention dilution, and how would you design around it?
10. Why is prompt injection difficult to defend against at the architectural level?

4.27 Summary

The transformer replaced sequential recurrence with self-attention, in which every position simultaneously queries every other position it is permitted to see. This removed both the sequential training bottleneck and the degradation of long-range information, and in doing so made it economically feasible to train models at internet scale.

A transformer block does two things: attention mixes information *across* positions, and a feed-forward network — the multi-layer perceptron of Chapter 3 — processes each position *independently*. Both are wrapped in residual connections and layer normalisation, without which deep stacks do not train. Attention itself is four steps: score every query against every key, scale by the square root of the key dimension, mask the future and apply softmax, then blend the values by the resulting weights. Multiple heads run this in parallel with separate projections so that several kinds of relevance can be tracked at once. Positional information must be injected explicitly, because a weighted sum is order-blind.

Causal masking creates the asymmetry that governs the economics of the whole platform: training sees all positions at once and is massively parallel, while generation must proceed one token at a time. Hence prefill and decode, hence time-to-first-token versus time-per-output-token, hence input tokens costing less than output tokens, and hence the KV cache — which, because keys and values depend causally on everything before them, is why API prompt caching works only on a prefix.

Finally, attention compares every position with every other, so its cost grows with the square of sequence length while each position's fixed unit of attention is spread ever more thinly. That single geometric fact is the entire technical case for retrieval, compaction, caching and deliberate placement — the disciplines of Part 10, and the reason the most tempting answer in Domain 5, *"just use the million-token window"*, is reliably the wrong one.

KEY TAKEAWAYS

1. **Attention lets every position read every earlier position directly** — no chain, no decay, fully parallel.
2. **A block is: mix across positions, then process each position.** Everything else is scaffolding you already knew.
3. `softmax(QKᵀ/√d)V` — four steps: score, scale, mask-and-normalise, blend.
4. **Causal masking** is why training parallelises and generation does not.
5. **Multi-head attention** tracks several kinds of relevance at once, at roughly no extra cost.
6. **Positional encoding is mandatory** because attention is order-blind.
7. **Feed-forward layers hold most of the parameters**, and most of the stored knowledge.
8. **Prefill is parallel and cheap per token; decode is sequential and expensive per token.** This sets the whole pricing and latency model.
9. **Prompt caching works on prefixes only**, because of causal dependency, and accelerates prefill alone.
10. **Attention is quadratic and its budget is fixed**, which is why retrieval, compaction and placement beat a bigger window.

4.28 Glossary

Term	Definition
Attention head	One independent set of query, key and value projections within multi-head attention.
Attention weights	The post-softmax distribution describing how much each position attends to each other position.
Causal mask	A constraint preventing any position from attending to later positions.
Cross-attention	Attention from a decoder's positions to an encoder's outputs, in encoder–decoder models.
Decode	The generation phase: one sequential forward pass per output token.
Decoder-only	An architecture in which every position attends only to preceding positions. Claude is one.
Embedding	The vector representation assigned to a token before it enters the first block.
Encoder-only	An architecture with bidirectional attention, used for understanding tasks and retrieval embeddings.
Feed-forward network	The per-position MLP inside each block; holds most of the model's parameters.
Key	The vector a position offers for comparison against other positions' queries.
KV cache	Stored keys and values for already-processed positions, avoiding recomputation during decode.
Layer normalisation	Rescaling activations within a layer to stabilise deep-network training.
Multi-head attention	Several attention operations in parallel with separate projections, concatenated and projected.

Term	Definition
Positional encoding	Information about token order added to embeddings; required because attention is order-blind.
Prefill	The phase in which the whole prompt is processed in one parallel pass, populating the KV cache.
Prompt caching	Reusing a stored KV cache for a repeated prompt prefix, reducing prefill cost and latency.
Query	The vector describing what a position is looking for.
Residual connection	Adding a sub-layer's input to its output, giving gradients a direct backward path.
RoPE	Rotary positional embedding; encodes relative position by rotating queries and keys.
Scaled dot-product attention	The standard mechanism: $\mathrm{softmax}(QK^T/\sqrt{d})V$.
Self-attention	Attention in which queries, keys and values all derive from the same sequence.
Transformer block	One unit of attention plus feed-forward network, with residuals and normalisation.
Unembedding	The final projection from model width to one score per vocabulary token.
Value	The content a position contributes when it is attended to.

4.29 Further Reading

Primary and canonical sources.

- Vaswani et al. (2017), *Attention Is All You Need*. The original paper. Now that you have the mechanism, it is readable in an evening.
- Alammar, *The Illustrated Transformer*. The best-known visual walkthrough; a useful complement to Figure 4.2.
- Karpathy, *Let's build GPT: from scratch, in code, spelled out*. The direct extension of the mini project in section 4.13.
- Su et al. (2021), *RoFormer: Enhanced Transformer with Rotary Position Embedding*. The positional scheme behind modern long-context models.
- Liu et al. (2023), *Lost in the Middle: How Language Models Use Long Contexts*. The empirical basis for the placement advice in this chapter and in Chapter 55.
- Claude Platform documentation, *Prompt caching* — the API surface for the mechanism in section 4.9, including current minimum cacheable lengths.

Where to go next in this book.

Chapter 5 examines attention in greater depth — the encoder–decoder split, cross-attention, and the interpretability findings on what heads actually learn. Chapter 6 then makes tokens and embeddings concrete: how the tokenizer is built, why token counts differ by language, and what an embedding vector really represents. Part 1 closes with Chapters 7 and 8 on training, alignment, RLHF, Constitutional AI,

hallucination and safety — after which Part 2 moves from how models are built to how they behave.